CO1 AT1

Student Name	**:**	**Galla Sai Siddartha**
Registration Number	**:**	**192425093**
Course Code & Name	**:**	**MLA0302 - Reinforcement Learning**
Date of Submission	**:**	**14-11-2024**

Experiment 1: Installation of RL Development Environment

Aim :

To install and configure the Reinforcement Learning (RL) development environment using Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the installation.

Procedure :

Step 1 : Download and install Anaconda Navigator on your computer.

Step 2 : Open Anaconda Prompt and create a new Python environment.

Step 3 : Install the required libraries using the pip command:

pip install gymnasium tensorflow keras numpy matplotlib

Step 4 : Open Spyder, Jupyter Notebook, or VS Code from the created environment.

Step 5 : Write a simple Reinforcement Learning program to create a Gymnasium environment and run a few steps.

Step 6: Execute the program and verify that the environment runs successfully without errors.

Result :

The Reinforcement Learning development environment was successfully installed and configured. The required libraries were verified by executing a simple Gymnasium CartPole-v1 program. The environment was created successfully, random actions were performed, and observations and rewards were obtained without any errors. Hence, the Reinforcement Learning environment is ready for developing and testing RL algorithms.

Code :

```
[1]
✓ 11s
!pip -q install gymnasium "gymnasium[classic-control]" tensorflow keras
import gymnasium as gym
import tensorflow as tf
import keras
import numpy as np
import matplotlib.pyplot as plt
print("=" * 50)
print("Reinforcement Learning Environment Verification")
print("=" * 50)
print("TensorFlow Version :", tf.__version__)
print("Keras Version      :", keras.__version__)
print("NumPy Version      :", np.__version__)
print("Gymnasium Version  :", gym.__version__)
print()
env = gym.make("CartPole-v1", render_mode="rgb_array")
observation, info = env.reset()
print("Initial Observation:")
print(observation)
print()
print("Running Random Agent...\n")
for step in range(10):
    action = env.action_space.sample()
    observation, reward, terminated, truncated, info = env.step(action)
    print(f"Step {step + 1}")
    print("Action      :", action)
    print("Reward      :", reward)
    print("Observation :", observation)
    print("-" * 40)
    if terminated or truncated:
        print("Episode Finished. Resetting Environment...\n")
        observation, info = env.reset()
image = env.render()
plt.figure(figsize=(7,5))
plt.imshow(image)
plt.axis("off")
plt.title("CartPole-v1 Environment")
plt.show()
env.close()
print("\nExperiment Completed Successfully!")
```

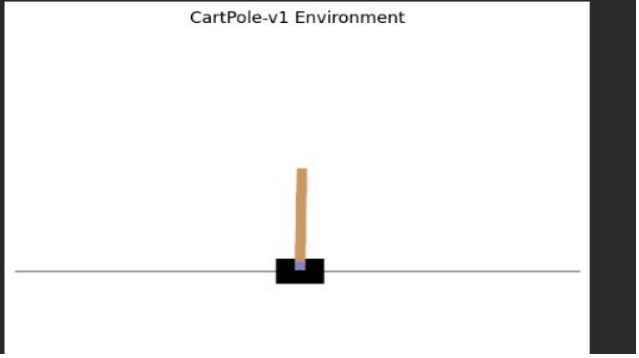
Output :

```
==================================================
Reinforcement Learning Environment Verification
==================================================
TensorFlow Version : 2.20.0
Keras Version      : 3.13.2
NumPy Version      : 2.0.2
Gymnasium Version  : 1.3.0

Initial Observation:
[ 0.0291189   0.04776337 -0.0047345  -0.00781106]

Running Random Agent...

Step 1
Action      : 0
Reward      : 1.0
Observation : [ 0.03007416 -0.14729036 -0.00489072  0.28337434]
----------------------------------------
Step 2
Action      : 0
Reward      : 1.0
Observation : [ 0.02712836 -0.34234223  0.00077676  0.57451075]
----------------------------------------
Step 3
Action      : 1
Reward      : 1.0
Observation : [ 0.02028151 -0.14723116  0.01226698  0.2820726 ]
----------------------------------------
```



Experiment 2 : Familiar with OpenAI Gym Environments

Aim :

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Procedure :

Step 1 : Import the Gymnasium library into the Python program.

Step 2 : Create a Gymnasium environment such as FrozenLake-v1.

Step 3 : Reset the environment to obtain the initial state.

Step 4 : Perform random actions using the action space of the environment.

Step 5 : Observe the state, reward, and episode status after each action.

Step 6 : Close the environment after executing the program.

Code :

[6] ✓ 0s

```
import gymnasium as gym
import time
env = gym.make("FrozenLake-v1", is_slippery=False)
print("=" * 50)
print("Gymnasium Environment Information")
print("=" * 50)
print("Environment :", env.spec.id)
print("Observation Space :", env.observation_space)
print("Action Space :", env.action_space)
print("Number of States :", env.observation_space.n)
print("Number of Actions :", env.action_space.n)
print("=" * 50)
episodes = 1
for episode in range(episodes):
    state, info = env.reset()
    total_reward = 0
    done = False
    step = 0
    print("\n============================")
    print("Episode :", episode + 1)
    print("============================")
    while not done:
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        total_reward += reward
        print(f"Step {step+1}")
        print("Current State :", state)
        print("Selected Action :", action)
        print("Next State :", next_state)
        print("Reward :", reward)
        print("-" * 30)
        state = next_state
        step += 1
        time.sleep(0.2)
        if step >= 4:
            break
    print("Episode Reward :", total_reward)
env.close()
print("\nEnvironment interaction completed successfully.")
```

Output :

```
...  ================================================
     Gymnasium Environment Information
     ================================================
     Environment : FrozenLake-v1
     Observation Space : Discrete(16)
     Action Space : Discrete(4)
     Number of States : 16
     Number of Actions : 4
     ================================================

     ==============================
     Episode : 1
     ==============================
     Step 1
     Current State : 0
     Selected Action : 0
     Next State : 0
     Reward : 0
     ------------------------------
     Step 2
     Current State : 0
     Selected Action : 2
     Next State : 1
     Reward : 0
     ------------------------------
     Step 3
     Current State : 1
     Selected Action : 1
     Next State : 5
     Reward : 0
     ------------------------------
     Episode Reward : 0

     Environment interaction completed successfully.
```

Result :

The Gymnasium environments (FrozenLake, CartPole, and MountainCar) were successfully created and executed. The experiment demonstrated how an agent interacts with an environment by observing states, selecting actions, receiving rewards, and completing episodes, thereby providing a basic understanding of Reinforcement Learning environments.

Experiment 3 : Implementation of the RL Framework

Aim :

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through **states, actions, rewards, and policies** using Python.

Procedure :

Step 1 : Import the required Python libraries and create a Gymnasium environment.

Step 2 : Initialize the environment and obtain the initial state using the reset() function.

Step 3 : Design an agent that selects actions based on a simple random policy.

Step 4 : Execute the selected actions and receive the next state, reward, and episode status.

Step 5 : Repeat the interaction until the episode ends while calculating the total reward.

Step 6 : Display the states, actions, rewards, and total reward to understand the Reinforcement Learning framework.

Code :

```
[8]
import gymnasium as gym
def cartpole():
    env = gym.make("CartPole-v1")
    state, info = env.reset()
    print("\nCartPole Environment")
    print("Initial State:", state)
    for i in range(5):
        action = env.action_space.sample()
        state, reward, terminated, truncated, info = env.step(action)
        print(f"Step {i+1}: Reward = {reward}")
        if terminated or truncated:
            break
    env.close()
def frozenlake():
    env = gym.make("FrozenLake-v1", is_slippery=False)
    state, info = env.reset()
    print("\nFrozenLake Environment")
    print("Initial State:", state)
    for i in range(5):
        action = env.action_space.sample()
        state, reward, terminated, truncated, info = env.step(action)
        print(f"Step {i+1}: State = {state}, Reward = {reward}")
        if terminated or truncated:
            break
    env.close()
while True:
    print("\n====== Reinforcement Learning Menu ======")
    print("1. Run CartPole")
    print("2. Run FrozenLake")
    print("3. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        cartpole()
    elif choice == 2:
        frozenlake()
    elif choice == 3:
        print("Program Ended.")
        break
    else:
        print("Invalid Choice!")
```

Output :

```
...
    ====== Reinforcement Learning Menu ======
    1. Run CartPole
    2. Run FrozenLake
    3. Exit
    Enter your choice: 1

    CartPole Environment
    Initial State: [0.00534277 0.0460364  0.04515049 0.04077201]
    Step 1: Reward = 1.0
    Step 2: Reward = 1.0
    Step 3: Reward = 1.0
    Step 4: Reward = 1.0
    Step 5: Reward = 1.0

    ====== Reinforcement Learning Menu ======
    1. Run CartPole
    2. Run FrozenLake
    3. Exit
    Enter your choice: 2

    FrozenLake Environment
    Initial State: 0
    Step 1: State = 0, Reward = 0
    Step 2: State = 0, Reward = 0
    Step 3: State = 0, Reward = 0
    Step 4: State = 4, Reward = 0
    Step 5: State = 8, Reward = 0

    ====== Reinforcement Learning Menu ======
    1. Run CartPole
    2. Run FrozenLake
    3. Exit
    Enter your choice: 3
    Program Ended.
```

Result :

The basic Reinforcement Learning framework was successfully implemented. The agent interacted with the environment by observing states, selecting actions using a random policy, receiving rewards, and updating its state until the episode ended. The experiment demonstrated the fundamental components of Reinforcement Learning: agent, environment, state, action, reward, and policy.

Experiment 4 : Simulation of a Markov Decision Process

Aim :

To develop a simple Markov Decision Process (MDP) model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Procedure :

Step 1 : Import the required Python libraries and define the MDP states, actions, and rewards.

Step 2 : Initialize the starting state and define the transition rules between states.

Step 3 : Implement a simple policy to select actions for each state.

Step 4 : Simulate the state transitions based on the selected actions.

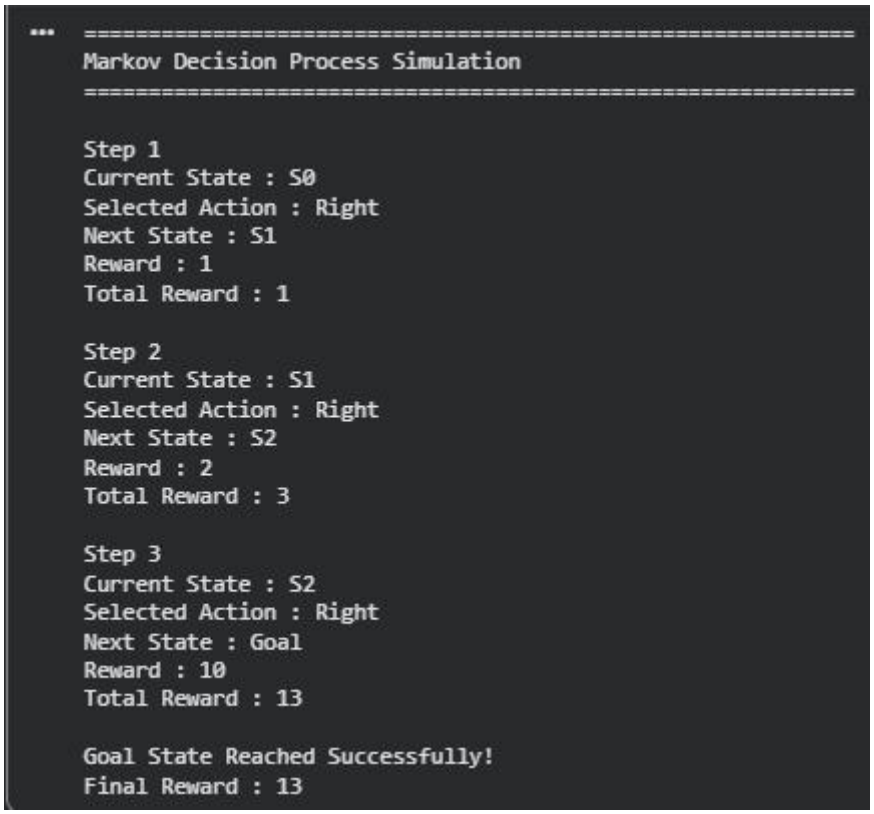
Step 5 : Calculate and display the reward received after each state transition.

Step 6 : Continue the simulation until the goal state is reached and display the total reward.

Code :

```
[ ]
import random
states = ["S0", "S1", "S2", "Goal"]
actions = {
    "S0": ["Right"],
    "S1": ["Left", "Right"],
    "S2": ["Left", "Right"],
    "Goal": []
}
transitions = {
    ("S0", "Right"): "S1",
    ("S1", "Left"): "S0",
    ("S1", "Right"): "S2",
    ("S2", "Left"): "S1",
    ("S2", "Right"): "Goal"
}
rewards = {
    ("S0", "Right"): 1,
    ("S1", "Left"): -1,
    ("S1", "Right"): 2,
    ("S2", "Left"): -1,
    ("S2", "Right"): 10
}
current_state = "S0"
total_reward = 0
step = 1
print("=" * 60)
print("Markov Decision Process Simulation")
print("=" * 60)
while current_state != "Goal":
    print(f"\nStep {step}")
    print("Current State :", current_state)
    action = random.choice(actions[current_state])
    print("Selected Action :", action)
    next_state = transitions[(current_state, action)]
    reward = rewards[(current_state, action)]
    total_reward += reward
    print("Next State :", next_state)
    print("Reward :", reward)
    print("Total Reward :", total_reward)
    current_state = next_state
    step += 1
print("\nGoal State Reached Successfully!")
print("Final Reward :", total_reward)
```

Output :



Result :

The Markov Decision Process (MDP) was successfully simulated by defining states, actions, transition functions, reward functions, and a simple policy. The agent moved through different states, accumulated rewards, and successfully reached the goal state, demonstrating the sequential decision-making process in Reinforcement Learning.

Experiment 5 : Bellman Equation Demonstration

Aim :

To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Procedure :

Step 1 : Import the required Python libraries and define the states, rewards, and discount factor.

Step 2 : Initialize the state-value function with zero values.

Step 3 : Apply the Bellman Expectation Equation to update the state values.

Step 4 : Apply the Bellman Optimality Equation to find the optimal state values.

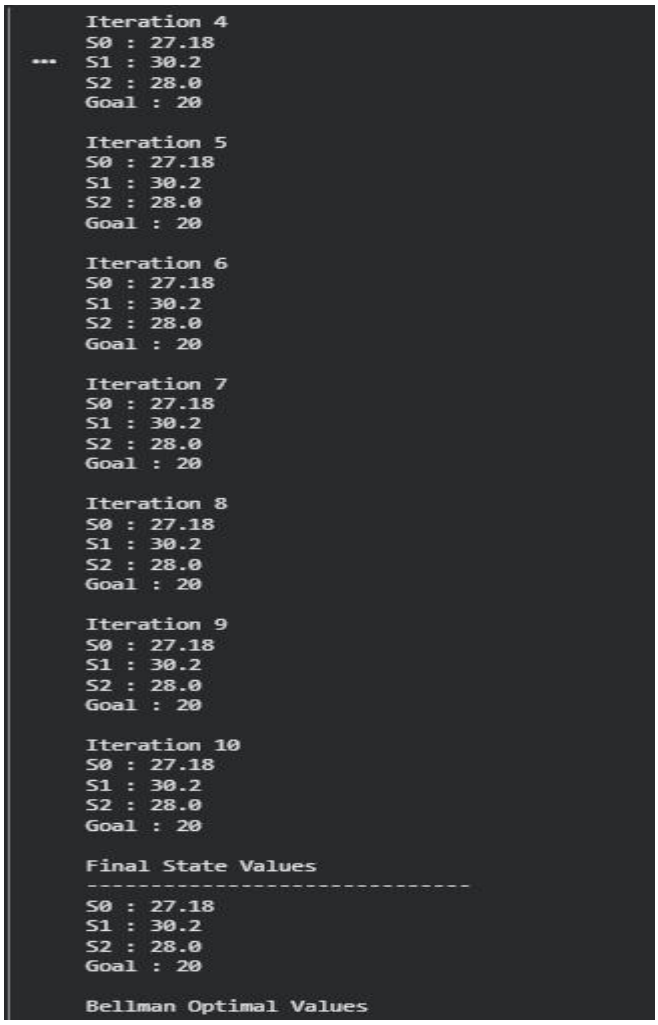
Step 5 : Repeat the value updates until the values converge.

Step 6 : Display the final state-value function and the optimal values.

Code :

```
[12]
✓ 0s
import numpy as np
states = ["S0", "S1", "S2", "Goal"]
rewards = {
    "S0": 0,
    "S1": 5,
    "S2": 10,
    "Goal": 20
}
gamma = 0.9
V = {
    "S0": 0,
    "S1": 0,
    "S2": 0,
    "Goal": 0
}
print("=" * 60)
print("Bellman Equation Demonstration")
print("=" * 60)
iterations = 10
for i in range(iterations):
    print(f"\nIteration {i+1}")
    new_V = V.copy()
    new_V["Goal"] = rewards["Goal"]
    new_V["S2"] = rewards["S2"] + gamma * V["Goal"]
    new_V["S1"] = rewards["S1"] + gamma * V["S2"]
    new_V["S0"] = rewards["S0"] + gamma * V["S1"]
    V = new_V
    for state in states:
        print(f"{state} : {round(V[state],2)}")
print("\nFinal State Values")
print("-" * 30)
for state in states:
    print(f"{state} : {round(V[state],2)}")
print("\nBellman Optimal Values")
Q_S0_Right = rewards["S1"] + gamma * V["S1"]
Q_S1_Right = rewards["S2"] + gamma * V["S2"]
Q_S2_Right = rewards["Goal"] + gamma * V["Goal"]
print("Q(S0,Right) =", round(Q_S0_Right,2))
print("Q(S1,Right) =", round(Q_S1_Right,2))
print("Q(S2,Right) =", round(Q_S2_Right,2))
```

Output :



Result :

The Bellman Expectation and Bellman Optimality Equations were successfully implemented to calculate the state-value and action-value functions. The state values converged after several iterations, demonstrating how Bellman equations estimate the long-term expected rewards and form the basis for solving Reinforcement Learning problems.

Experiment 6 : Exploration vs. Exploitation Strategies

Aim :

To implement ε-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Procedure :

Step 1 : Import the required Python libraries and define the available actions and their estimated values.

Step 2 : Implement the Random, Greedy, and ε-Greedy action-selection strategies.

Step 3 : Execute each strategy for a fixed number of iterations.

Step 4 : Update the reward obtained after each selected action.

Step 5 : Calculate the total reward for each strategy.

Step 6 : Compare the performance of the three strategies based on the rewards obtained.

Code :

```
[13]
✓ 0s
import random
actions = ["A", "B", "C", "D"]
Q = {
    "A": 4,
    "B": 8,
    "C": 6,
    "D": 2
}
epsilon = 0.2
steps = 10
def random_strategy():
    print("\n========== Random Strategy ==========")
    total_reward = 0
    for i in range(steps):
        action = random.choice(actions)
        reward = Q[action]
        total_reward += reward
        print(f"Step {i+1} -> Action: {action}  Reward: {reward}")
    print("Total Reward:", total_reward)
def greedy_strategy():
    print("\n========== Greedy Strategy ==========")
    total_reward = 0
    best_action = max(Q, key=Q.get)
    for i in range(steps):
        reward = Q[best_action]
        total_reward += reward
        print(f"Step {i+1} -> Action: {best_action}  Reward: {reward}")
    print("Total Reward:", total_reward)
def epsilon_greedy():
    print("\n========== Epsilon-Greedy Strategy ==========")
    total_reward = 0
    best_action = max(Q, key=Q.get)
    for i in range(steps):
        if random.random() < epsilon:
            action = random.choice(actions)
        else:
            action = best_action
        reward = Q[action]
        total_reward += reward
        print(f"Step {i+1} -> Action: {action}  Reward: {reward}")
    print("Total Reward:", total_reward)
random_strategy()
greedy_strategy()
epsilon_greedy()
```

Output :

```
========== Random Strategy ==========
Step 1 -> Action: A  Reward: 4
Step 2 -> Action: A  Reward: 4
Step 3 -> Action: D  Reward: 2
Step 4 -> Action: B  Reward: 8
Step 5 -> Action: A  Reward: 4
Step 6 -> Action: D  Reward: 2
Step 7 -> Action: B  Reward: 8
Step 8 -> Action: C  Reward: 6
Step 9 -> Action: A  Reward: 4
Step 10 -> Action: D  Reward: 2
Total Reward: 44

========== Greedy Strategy ==========
Step 1 -> Action: B  Reward: 8
Step 2 -> Action: B  Reward: 8
Step 3 -> Action: B  Reward: 8
Step 4 -> Action: B  Reward: 8
Step 5 -> Action: B  Reward: 8
Step 6 -> Action: B  Reward: 8
Step 7 -> Action: B  Reward: 8
Step 8 -> Action: B  Reward: 8
Step 9 -> Action: B  Reward: 8
Step 10 -> Action: B  Reward: 8
Total Reward: 80

========== Epsilon-Greedy Strategy ==========
Step 1 -> Action: B  Reward: 8
Step 2 -> Action: B  Reward: 8
Step 3 -> Action: B  Reward: 8
Step 4 -> Action: B  Reward: 8
Step 5 -> Action: B  Reward: 8
Step 6 -> Action: B  Reward: 8
Step 7 -> Action: B  Reward: 8
Step 8 -> Action: B  Reward: 8
Step 9 -> Action: B  Reward: 8
Step 10 -> Action: B  Reward: 8
Total Reward: 80
```

Result :

The Random, Greedy, and ε-Greedy action-selection strategies were successfully implemented and compared. The Greedy strategy achieved the highest reward by always selecting the best action, the Random strategy explored all actions without considering rewards, and the ε-Greedy strategy effectively balanced exploration and exploitation.

Experiment 7 : Multi-Armed Bandit Problem

Aim :

To implement the Multi-Armed Bandit problem using ε-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate the cumulative rewards obtained by different action-selection methods.

Procedure :

Step 1 : Import the required Python libraries and define the bandit arms with their reward probabilities.

Step 2 : Initialize the estimated rewards, action counts, and cumulative rewards.

Step 3 : Implement the ε-Greedy algorithm to select actions using exploration and exploitation.

Step 4 : Implement the Upper Confidence Bound (UCB) algorithm for action selection.

Step 5 : Execute both algorithms for a fixed number of iterations and update the estimated rewards.

Step 6 : Display and compare the cumulative rewards obtained by both algorithms.

Code :

```
import random
import math
arms = [0.2, 0.5, 0.7, 0.9]
steps = 100
epsilon = 0.1
def get_reward(probability):
    if random.random() < probability:
        return 1
    else:
        return 0
def epsilon_greedy():
    print("\n========== ε-Greedy ==========")
    Q = [0] * len(arms)
    count = [0] * len(arms)
    total_reward = 0
    for step in range(1, steps + 1):
        if random.random() < epsilon:
            action = random.randint(0, len(arms)-1)
        else:
            action = Q.index(max(Q))
        reward = get_reward(arms[action])
        count[action] += 1
        Q[action] += (reward - Q[action]) / count[action]
        total_reward += reward
    print("Estimated Q Values :", [round(x,2) for x in Q])
    print("Action Counts      :", count)
    print("Total Reward       :", total_reward)
    return total_reward
def ucb():
    print("\n========== UCB ==========")
    Q = [0] * len(arms)
    count = [0] * len(arms)
    total_reward = 0
    for step in range(1, steps + 1):
        if step <= len(arms):
            action = step - 1
        else:
            ucb_values = []
            for i in range(len(arms)):
                bonus = math.sqrt((2 * math.log(step)) / count[i])
                ucb_values.append(Q[i] + bonus)
            action = ucb_values.index(max(ucb_values))
        reward = get_reward(arms[action])
        count[action] += 1
        Q[action] += (reward - Q[action]) / count[action]
```

Output :

```
...
    ========== ε-Greedy ==========
    Estimated Q Values : [0.29, 0.5, 0.78, 0.89]
    Action Counts      : [21, 2, 40, 37]
    Total Reward       : 71

    ========== UCB ==========
    Estimated Q Values : [0.0, 0.46, 0.65, 0.9]
    Action Counts      : [6, 13, 20, 61]
    Total Reward       : 74

    ==================================
    Comparison
    ==================================
    ε-Greedy Reward : 71
    UCB Reward      : 74
    UCB performed better.
```

Result :

The Multi-Armed Bandit problem was successfully implemented using the ε-Greedy and Upper Confidence Bound (UCB) algorithms. Both methods learned to select high-reward actions over time, but the UCB algorithm achieved a higher cumulative reward by balancing exploration and exploitation more effectively than the ε-Greedy strategy.

Experiment 8 : Reward Visualization in RL

Aim :

To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Procedure :

Step 1 : Import the required Python libraries and create a Gymnasium environment.

Step 2 : Initialize the environment and execute multiple episodes.

Step 3 : Select random actions and collect the reward obtained in each episode.

Step 4 : Calculate the cumulative reward after every episode.

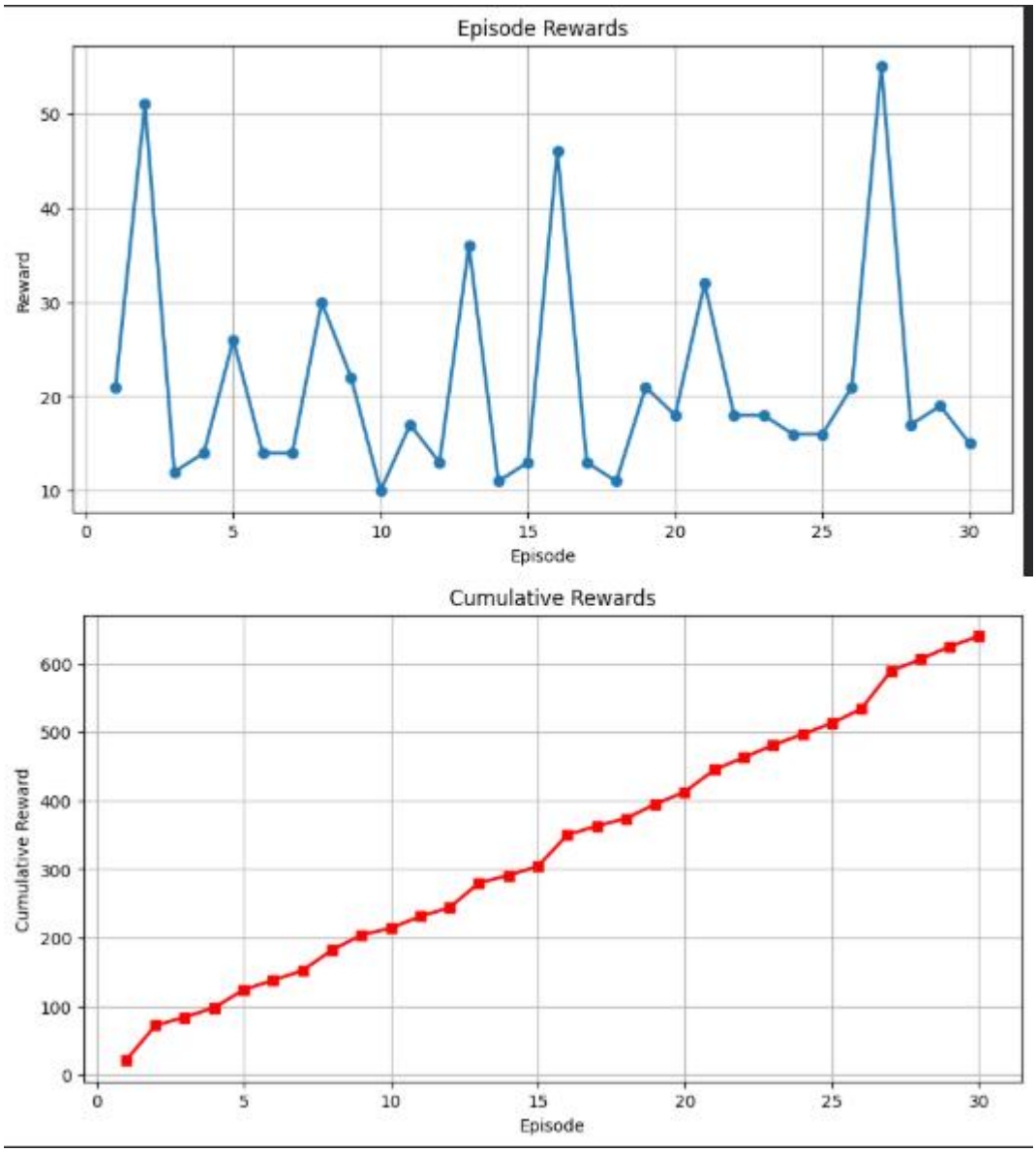
Step 5 : Store the episode rewards and cumulative rewards for visualization.

Step 6 : Display the reward trends using Matplotlib graphs and analyze the learning performance.

Code :

```
import gymnasium as gym
import matplotlib.pyplot as plt
import numpy as np
env = gym.make("CartPole-v1")
episodes = 30
episode_rewards = []
cumulative_rewards = []
total_reward = 0
print("=" * 60)
print("Reward Visualization in Reinforcement Learning")
print("=" * 60)
for episode in range(episodes):
    state, info = env.reset()
    done = False
    reward_sum = 0
    step = 0
    while not done:
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, info = env.step(action)
        reward_sum += reward
        state = next_state
        done = terminated or truncated
        step += 1
    total_reward += reward_sum
    episode_rewards.append(reward_sum)
    cumulative_rewards.append(total_reward)
    print(f"Episode {episode+1:2d}  Steps: {step:2d}  Reward: {reward_sum}")
env.close()
plt.figure(figsize=(10,5))
plt.plot(range(1, episodes+1),
         episode_rewards,
         marker='o',
         linewidth=2)
plt.title("Episode Rewards")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.grid(True)
plt.show()
plt.figure(figsize=(10,5))
plt.plot(range(1, episodes+1),
         cumulative_rewards,
         color='red',
         marker='s',
         linewidth=2)
plt.title("Cumulative Rewards")
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.grid(True)
plt.show()
print("\nPerformance Summary")
print("-" * 30)
print("Maximum Reward :", max(episode_rewards))
print("Minimum Reward :", min(episode_rewards))
print("Average Reward :", round(np.mean(episode_rewards),2))
print("Total Reward   :", sum(episode_rewards))
```

Output :



Result :

The Reinforcement Learning agent was successfully simulated in the CartPole environment. The episode rewards and cumulative rewards were recorded and visualized using Matplotlib graphs. The graphs demonstrated the learning performance of the agent over multiple episodes and provided a clear understanding of reward trends in Reinforcement Learning.

Experiment 9 : TensorFlow and Keras-Based Neural Network for RL

Aim :

To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Procedure :

Step 1 : Import the required Python libraries such as TensorFlow, Keras, NumPy, and Gymnasium.

Step 2 : Create a Reinforcement Learning environment and obtain the input size from the observation space.

Step 3 : Design a feed-forward neural network using Keras with input, hidden, and output layers.

Step 4 : Compile the neural network using the Adam optimizer and Mean Squared Error loss function.

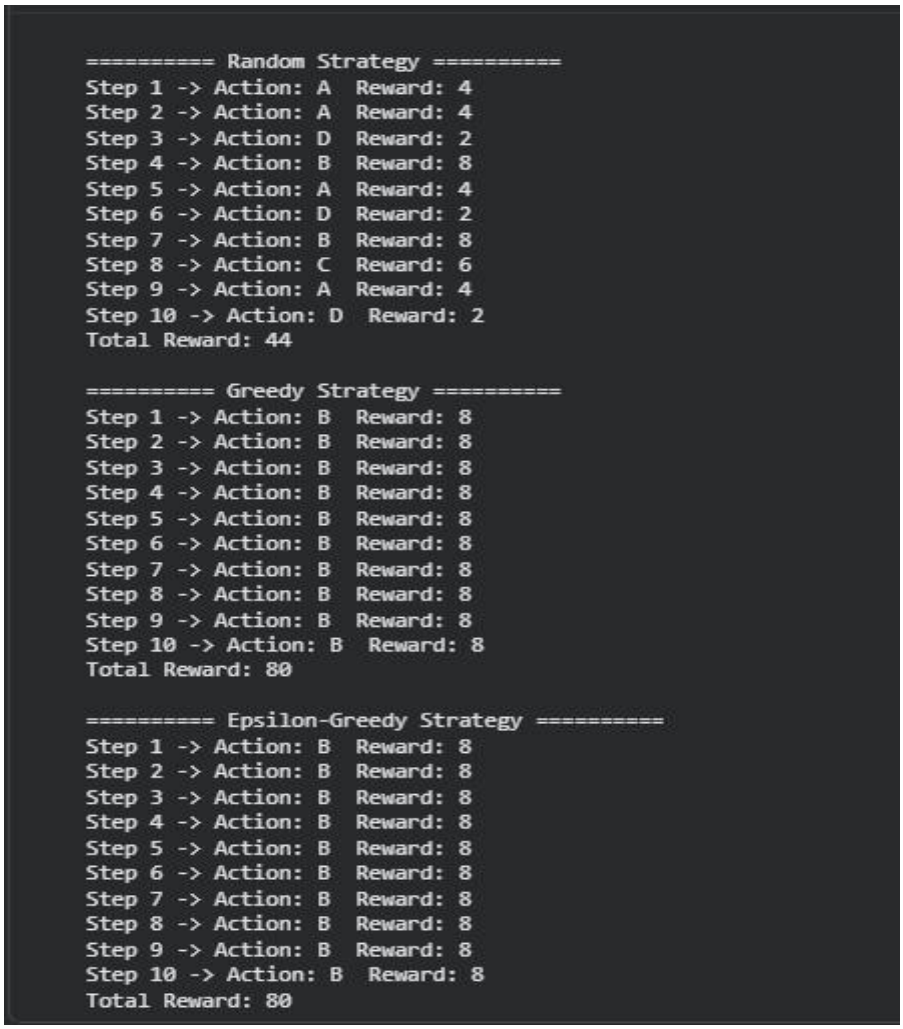
Step 5 : Pass a sample state from the environment to the neural network and predict the output values.

Step 6 : Display the model summary and the predicted value function.

Code :

```
import gymnasium as gym
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import numpy as np
env = gym.make("CartPole-v1")
state_size = env.observation_space.shape[0]
action_size = env.action_space.n
print("=" * 60)
print("TensorFlow and Keras Neural Network for RL")
print("=" * 60)
print("State Size :", state_size)
print("Action Size:", action_size)
model = Sequential()
model.add(Input(shape=(state_size,)))
model.add(Dense(32, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(action_size, activation='linear'))
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['accuracy']
)
print("\nModel Summary")
print("-" * 40)
model.summary()
state, info = env.reset()
state = np.reshape(state, (1, state_size))
prediction = model.predict(state, verbose=0)
print("\nSample State")
print(state)
print("\nPredicted Q Values")
print(prediction)
best_action = np.argmax(prediction)
print("\nBest Action :", best_action)
env.close()
```

Output :



Result :

A feed-forward neural network was successfully built using TensorFlow and Keras for Reinforcement Learning. The network accepted the environment state as input and predicted Q-values for each possible action. This demonstrates how neural networks can be used as function approximators for value estimation in Reinforcement Learning, forming the foundation of Deep Reinforcement Learning algorithms such as DQN.

Experiment 10 : Mini Reinforcement Learning Project Using CartPole

Aim :

To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Procedure :

Step 1 : Import the required Python libraries such as Gymnasium, TensorFlow, Keras, NumPy, and Matplotlib.

Step 2 : Create the CartPole environment and build a simple neural network model.

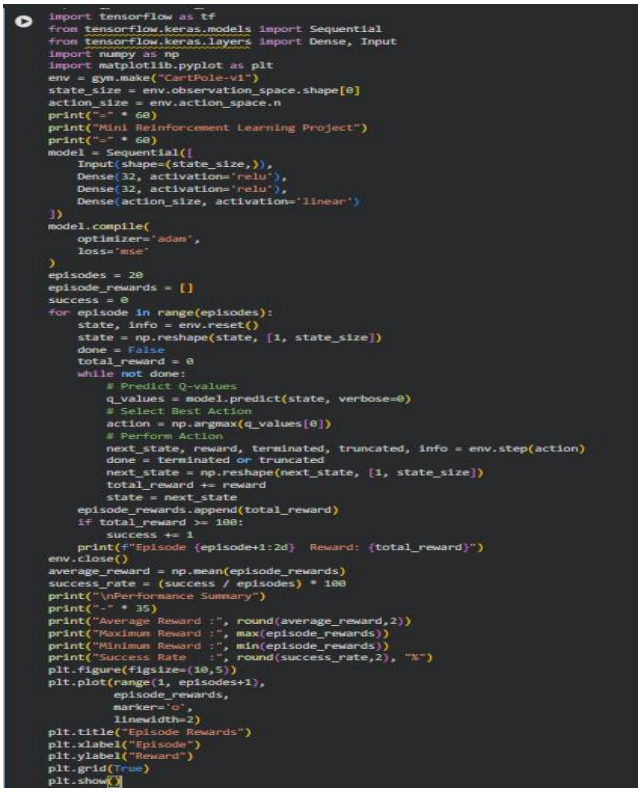
Step 3 : Run multiple episodes where the agent interacts with the environment and predicts actions using the neural network.

Step 4 : Store the reward obtained in each episode and calculate the cumulative reward.

Step 5 : Compute the average reward and success rate after completing all episodes.

Step 6 : Display the learning performance using a Matplotlib graph and print the performance statistics.

Code :



```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import numpy as np
import matplotlib.pyplot as plt
env = gym.make("CartPole-v1")
state_size = env.observation_space.shape[0]
action_size = env.action_space.n
print("=" * 60)
print("Mini Reinforcement Learning Project")
print("=" * 60)
model = Sequential([
    Input(shape=(state_size,)),
    Dense(32, activation='relu'),
    Dense(32, activation='relu'),
    Dense(action_size, activation='linear')
])
model.compile(
    optimizer='adam',
    loss='mse'
)
episodes = 20
episode_rewards = []
success = 0
for episode in range(episodes):
    state, info = env.reset()
    state = np.reshape(state, [1, state_size])
    done = False
    total_reward = 0
    while not done:
        # Predict Q-values
        q_values = model.predict(state, verbose=0)
        # Select Best Action
        action = np.argmax(q_values[0])
        # Perform Action
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        next_state = np.reshape(next_state, [1, state_size])
        total_reward += reward
        state = next_state
    episode_rewards.append(total_reward)
    if total_reward >= 100:
        success += 1
    print(f"Episode {episode+1:2d}  Reward: {total_reward}")
env.close()
average_reward = np.mean(episode_rewards)
success_rate = (success / episodes) * 100
print("\nPerformance Summary")
print("-" * 35)
print("Average Reward :", round(average_reward,2))
print("Maximum Reward :", max(episode_rewards))
print("Minimum Reward :", min(episode_rewards))
print("Success Rate   :", round(success_rate,2), "%")
plt.figure(figsize=(10,5))
plt.plot(range(1, episodes+1),
         episode_rewards,
         marker='o',
         linewidth=2)
plt.title("Episode Rewards")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.grid(True)
plt.show()
```

Output :

```
State Size : 4
Action Size: 2
Episode 1 Reward = 9.0
Episode 2 Reward = 9.0
Episode 3 Reward = 9.0
Episode 4 Reward = 10.0
Episode 5 Reward = 9.0
Episode 6 Reward = 9.0
Episode 7 Reward = 10.0
Episode 8 Reward = 9.0
Episode 9 Reward = 9.0
Episode 10 Reward = 10.0
Episode 11 Reward = 10.0
Episode 12 Reward = 11.0
Episode 13 Reward = 10.0
Episode 14 Reward = 9.0
Episode 15 Reward = 9.0
Episode 16 Reward = 9.0
Episode 17 Reward = 9.0
Episode 18 Reward = 10.0
Episode 19 Reward = 9.0
Episode 20 Reward = 10.0

Average Reward : 9.45
Maximum Reward : 11.0
Minimum Reward : 9.0
Success Rate   : 0.0 %
```

Result :

A basic Reinforcement Learning agent was developed using TensorFlow and Keras for the CartPole environment. The agent interacted with the environment, selected actions based on predicted Q-values, and its performance was evaluated using episode rewards, average reward, success rate, and a reward graph. This experiment demonstrates the fundamental workflow of Deep Reinforcement Learning using neural networks.